Dynamic Partial Reconfiguration

Arun Murugappan, Pranav T Varkey, Shrikrishna Pandit

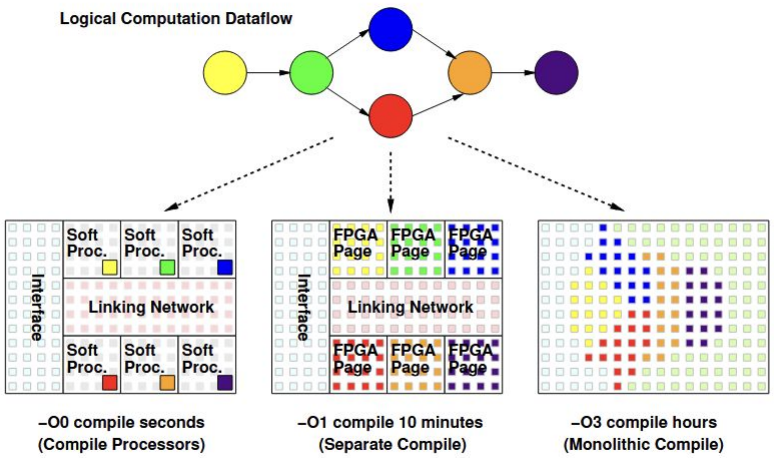
Table of contents

- Fast and Flexible FPGA Development using Hierarchical Partial Reconfiguration(2022)
- RapidStream 2.0: Automated Parallel Implementation of Latency–Insensitive FPGA Designs Through Partial Reconfiguration(2022, 2023)
- ZyPR: End-to-end Build Tool and Runtime Manager for Partial Reconfiguration of FPGA SoCs at the Edge (2022)

Toolflows

PLD: Fast FPGA Compilation to Make Reconfigurable Acceleration Compatible with Modern Incremental Refinement Software Development.

- This paper develops the methods on which the HPR paper works.
- It introduces 3 ways to compile a HLS code.



Toolflows - PLD

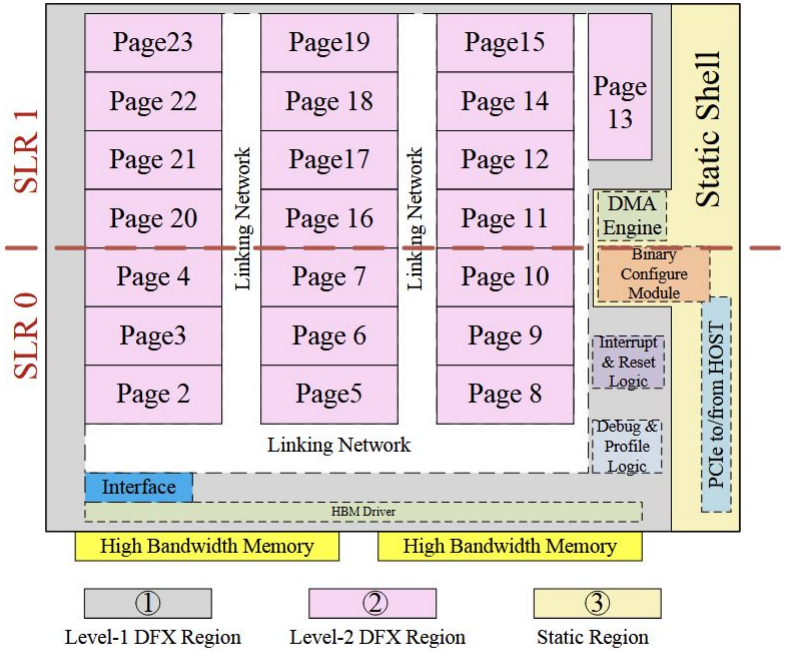


Figure 3: PLD FPGA Decomposition: Pages, Linking Network, and Support Infrastructure

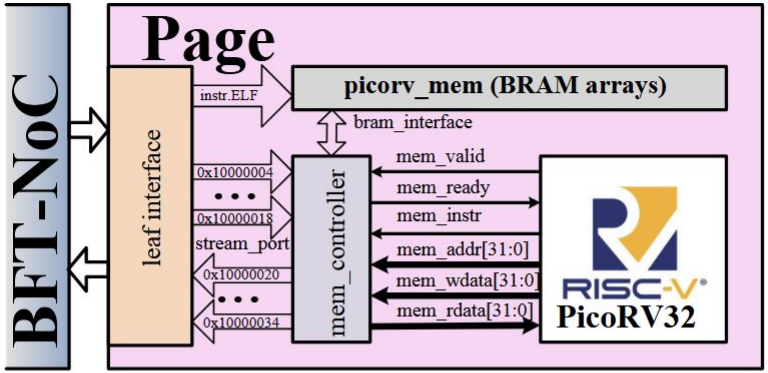


Figure 4: RISC-V Integration for One PR Page - mem_controller is responsible for connecting native memory interface, stream interface and BRAM access interface; processor instructions (ELF) are loaded from the BFT-NoC via the leaf interface.

Toolflows - PLD

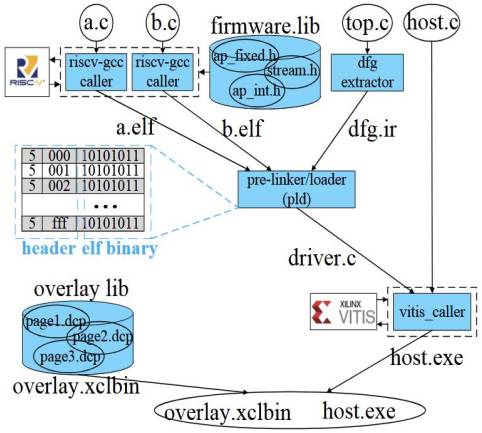


Figure 5: -O0 Compile Flow

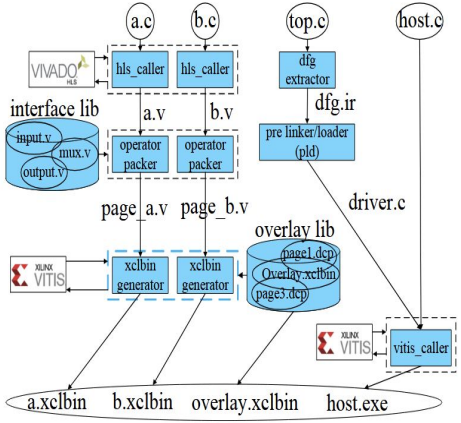


Figure 6: -O1 Compile Flow

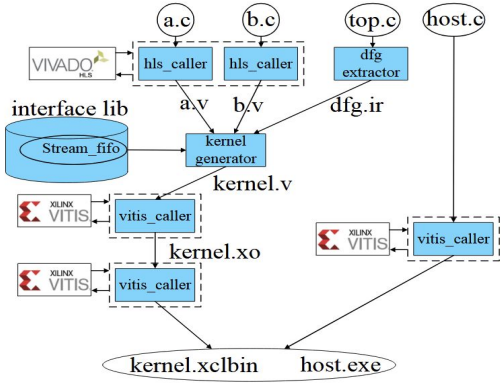


Figure 7: -O3 Compile Flow

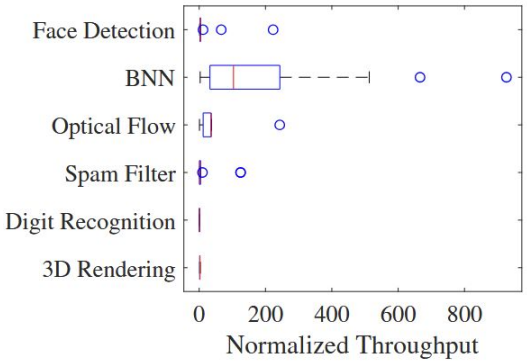


Figure 10: Speedup Distribution with One Softcore (`-O0`) and Rest on FPGA Pages (`-O1`) Compared to All Softcore (`-O0`)

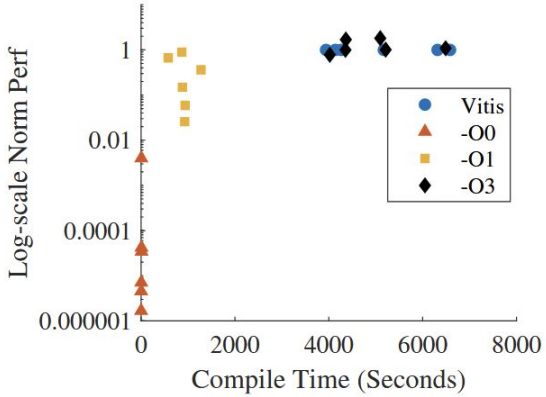


Figure 11: Performance vs. Compile Time

Table 2: Rosetta Benchmark Compile Time (in seconds)

	Vitis Flow Compile with 30 Threads					PLD -O3 Compile with 30 Threads					PLD -O1 Compile with 8 Threads for each Operator					PLD -O0 Compile
	hls	syn	p&r	bit	total	hls	syn	p&r	bit	total	hls	syn	p&r	bitgen	total	riscv g++
3D-rendering	104	1190	2364	606	**4264**	36	1306	2359	662	**4363**	35	119	294	130	**578**	**1.0**
Digit Recognition	144	1627	2673	729	**5173**	30	1701	2827	654	**5212**	29	114	560	164	**867**	**1.5**
Spam Filter	69	1308	1867	698	**3942**	20	1397	2243	695	**4355**	12	139	597	177	**925**	**3.1**
Optical Flow	84	1293	2094	668	**4139**	19	1531	2854	693	**5097**	18	102	583	177	**880**	**2.4**
Face Detection	542	1738	3280	728	**6288**	20	1219	2115	668	**4022**	24	212	545	158	**939**	**2.1**
Binary NN	485	2946	2430	723	**6584**	225	2153	3292	820	**6490**	225	415	372	140	**1152**	**3.4**

Table 3: Rosetta Benchmark Performance

	Vitis Flow		PLD -O3		PLD -O1		PLD -O0		X86 g++	Vitis Emu
	Fmax	per input	Fmax	per input	Fmax	per input	Fmax	per input	per input	per input
Rendering	300 MHz	1.6 ms	300 MHz	0.9 ms	200 MHz	1.4 ms	200 MHz	3 s	0.1 s	3 s
Digit Reg	300 MHz	10.5 ms	300 MHz	3.9 ms	200 MHz	6.2 ms	200 MHz	137 s	824.0 s	7400 s
Spam	300 MHz	18.6 ms	300 MHz	20.0 ms	200 MHz	68.7 ms	200 MHz	752 s	4.5 s	100 s
Optical	200 MHz	13.6 ms	300 MHz	4.8 ms	200 MHz	48.4 ms	200 MHz	10 935 s	7.3 s	151 s
Face	300 MHz	24.1 ms	300 MHz	31.0 ms	200 MHz	125.0 ms	200 MHz	527 s	17.5 s	352 s
BNN	150 MHz	5.1 ms	300 MHz	4.7 ms	200 MHz	7.1 ms	200 MHz	983 s	135.0 s	7100 s

Table 4: Rosetta Benchmark Area Consumption

	Vitis Flow			PLD -O3			PLD -O1				PLD -O0			
	LUT	B18	DSP	LUT	B18	DSP	LUT	B18	DSP	PAGE#	LUT	B18	DSP	PAGE#
3D-rendering	4225	64	13	17696	128	26	22823	106	18	6	119208	576	864	6
Digit Recognition	36070	382	1	50595	406	0	63923	441	0	20	393224	1680	2832	20
Spam Filter	9616	34	224	21011	126	256	50965	204	256	16	291480	1176	2088	16
Optical Flow	26974	136	158	27278	192	312	43231	211	312	16	313752	1296	2256	16
Face Detection	51549	156	97	127890	322	192	164385	296	145	20	393224	1680	2832	20
Binary NN	26724	46	5	44077	1130	5	64093	1197	4	22	437768	1920	3168	22

(B18: BRAM18)

Toolflows - PLD

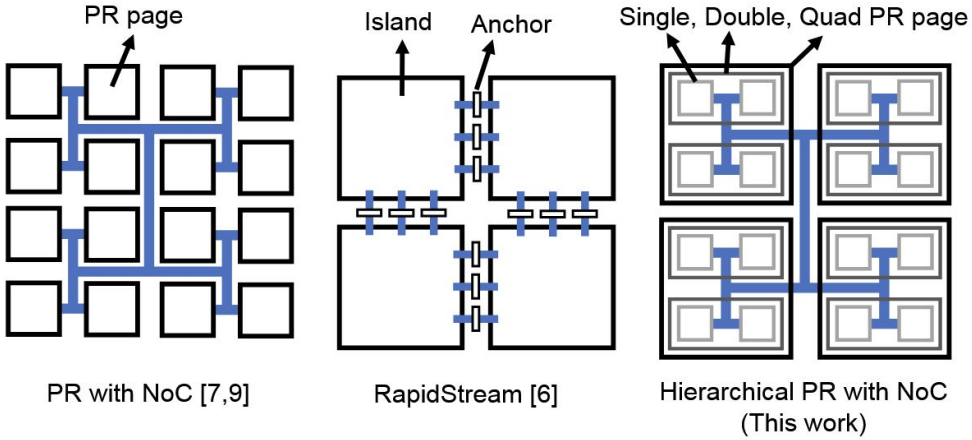
Rules of Decomposition:

- **Operator Discipline:**
  - Use **Streaming Computation**: Operator functions must be refined to utilize streaming computation.
  - **Streaming** over Global Shared Memory.
  - Avoid Dynamic Allocation or Recursion, Compatible Datatypes
  - Conditional Processor-Only Operations.
- **Operator Decomposition:**
  - Modular Computational Blocks.
  - Stream-Connected Communication,
  - Logical Decoupling
  - Optional for better performance
    - Merge Operators which otherwise suffer from I/O bottlenecks
    - Split Operators to exploit data parallelism

Toolflows

Fast and Flexible FPGA Development using Hierarchical Partial Reconfiguration:

- FPGA compilation is notoriously slow
- Existing solutions force users to break designs into small, fixed regions, often resulting in suboptimal performance or extra manual effort.
- Unlike prior approaches with rigid, fixed-sized regions, the method this paper follows, allows, flexible, hierarchical regions that adapt to the size of each design block.



Toolflows - Fast and flexible

Hierarchical Partial Reconfiguration (HPR) Framework:

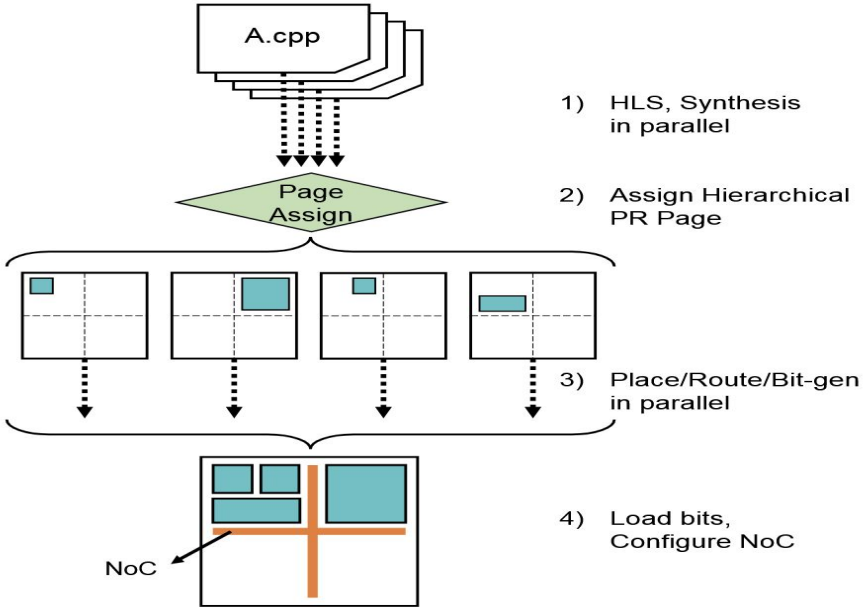


Fig. 2. Separate Compilation with Hierarchical PR Pages Overview

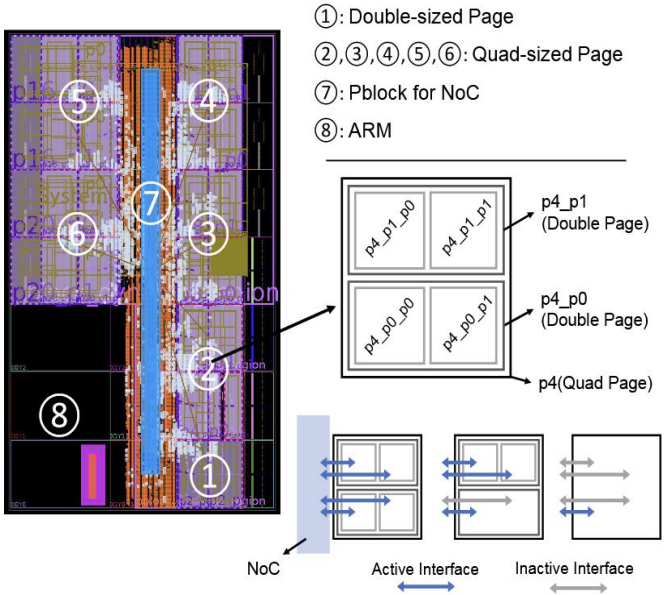
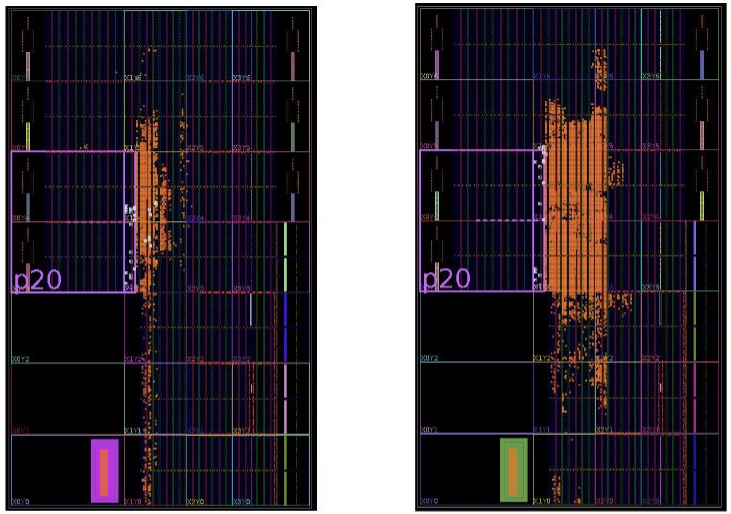


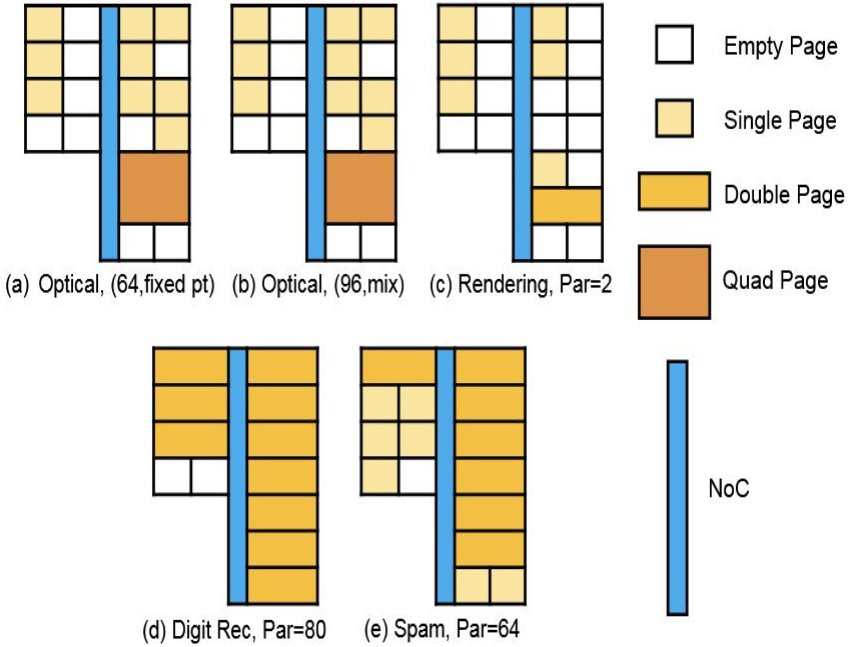
Fig. 3. Example Overlay on the Xilinx ZU9EG FPGA

Toolflows - Fast and flexible

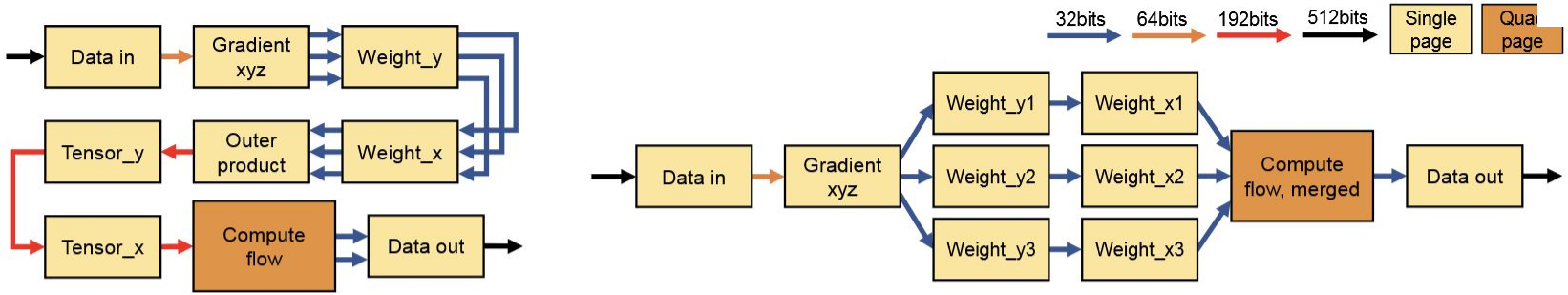


(a) p20 with p_NoC (b) p20 without p_NoC

Fig. 4. Abstract shells for p20 comparison. Orange elements are the static resources showing the extent of the expanded regions.



Toolflows - Fast and flexible



(a) Natural Decomposition of Optical Flow

(b) Decomposition of Optical Flow after Incremental Development

Fig. 5. Optical Flow Dataflow Graph

TABLE III
OPTICAL FLOW INCREMENTAL DEVELOPMENT WITH HIERARCHICAL PR PAGES

Steps	Largest Operator	Page Size	Compile Jobs	Largest Operator Resource Usage			Incremental Compile Time	App Latency
				LUTs	B18s†	DSPs		
1) Natural	Compute flow	Quad	9	16829	7	24	**261s**	18.7ms
2) Merge Tensor_x	Compute flow	Quad	1	17560	7	54	**245s**	18.1ms
3) Merge Tensor_y	Compute flow	Quad	1	18473	33	68	**274s**	16.0ms
4) Merge Outer prod.	Compute flow	Quad	1	20357	39	74	**288s**	14.7ms
5) Split Weight_x	Weight_x1	Single	3	1690	6	10	**94s**	14.7ms
6) Split Weight_y	Weight_y1	Single	3	1791	18	10	**107s**	14.7ms
7) Optimize Compute flow	Compute flow	Quad	1	20307	45	78	**291s**	8.7ms
8) Change data type	Compute flow	Quad	1	12471	49	192	**274s**	8.7ms

† B18s: BRAM18s

"App Latency" – application execution time per input

Toolflows - Fast and flexible

Aspect	**Prior Work (Fixed PR)**	**This Work (Hierarchical PR)**
Region Size	Fixed	Variable (Single/Double/Quad)
Manual Effort	High (manual split)	Low (automatic assignment)
Performance	Moderate	1.4–4.9× improvement
Compile Time	Hours	2–5 minutes (2.2–5.3× faster)
Flexibility	Low	High

- The Rosetta HLS benchmark was used to get the above results. The platform used for testing was the ZCU102 embedded platform according to the paper.